

14 | tokio实战：编写一个网络命令程序

唐刚 · Rust 语言从入门到实战



你好，我是 Mike，上一节课我们了解了 Rust 异步编程和 tokio 的基础知识，今天我们就来一起用 tokio 做一个小应用。

准备阶段

我们常常需要知道远程服务器上的一些信息，这有一些现成的工具可以做到。我们来试一下如何使用 tokio 实现这一功能。

目标：编写一个获取服务器时间的命令程序。

任务分解：

1. 命令行：这个工具取名为 `getinfo`，参数格式是 `getinfo {ip}`，就是在 `getinfo` 后接 IP 地址，获取服务器时间。
2. tcp server：监听 8888 端口，获取从客户端来的请求，然后获取服务器本地时间，返回。

3. tcp client: 连接服务端地址 `ip:port`, 向服务端发送获取服务器时间指令。

4. 测试。

实现

下面我们开始实现。

创建项目

我们打开终端或者 IDE 中的 Terminal, 执行:

```
1 cargo new --bin getinfo
```

 复制代码

命令行雏形

Rust 标准库中实际已经有获取命令行参数的功能, `std::env` 提供了一种获取命令行参数的方法 `std::env::args()`, 可以将命令行参数转换成一个迭代器, 通过 for 循环就可以遍历所有命令行参数, 当然也可以使用迭代器上的 `.nth()` 直接定位到某一个参数。比如:

```
1 let addr = env::args()
2   .nth(1)
3   .unwrap_or_else(|| "127.0.0.1:8888".to_string());
```

 复制代码

有了这个功能, 我们就可以得到命令行的初始版本。

```
1 use std::env;
2
3 fn main() {
4     let addr = env::args()
5       .nth(1)
6       .unwrap_or("127.0.0.1:8888".to_string());
7 }
```

 复制代码

```
8     println!("{}", addr);  
9 }
```

检查一下 Cargo.toml 中的配置，我们的应用名字应该叫 getinfo。

[复制代码](#)

```
1 [package]  
2 name = "getinfo"  
3 version = "0.1.0"  
4 edition = "2021"  
5  
6 # See more keys and their definitions at https://doc.rust-lang.org/cargo/referenc  
7  
8 [dependencies]
```

执行 cargo build 编译，会出现 target 目录。

[复制代码](#)

```
1 Cargo.lock Cargo.toml src/ target/
```

执行 cargo run 或 ./target/debug/getinfo。可以得到输出 127.0.0.1:8888。

而执行 cargo run -- 127.0.0.1:8000 或 ./target/debug/getinfo 127.0.0.1:8000，可以得到输出 127.0.0.1:8000。

这里我们分析一下这个命令行的形式。

[复制代码](#)

```
1 ./target/debug/getinfo ip_address
```

命令行参数从左到右按序号从 0 开始计数，上面命令中，./target/debug/getinfo 序号为 0，ip_address 部分序号就是 1，如果后面还有其他参数，那么序号依次递增。所以你就可以理解为什么我们上面的代码中，使用 .nth(1) 取 IP 地址的信息。

标准库中命令行相关的功能虽然比较初级，但是对于我们的例子来说，已经够用了。Rust 生态中有个非常好用的写命令行的库：[🔗 clap](#)，如果你想写一个功能丰富的命令程序，可以去尝试一下这个 clap。

下面我们就要开始 tokio tcp server 的创建。

添加依赖

先加入 tokio 的依赖，在项目目录下执行命令。

 复制代码

```
1 cargo add tokio --features full
2 cargo add tokio-util --features full
3 cargo add futures
4 cargo add bytes
```

执行完这几个添加依赖的命令后，Cargo.toml 文件现在看起来是类似下面这个样子：

 复制代码

```
1 mike@LAPTOP-04V0EV33:~/works/jikeshijian/getinfo$ cat Cargo.toml
2 [package]
3 name = "getinfo"
4 version = "0.1.0"
5 edition = "2021"
6
7 # See more keys and their definitions at https://doc.rust-lang.org/cargo/referenc
8
9 [dependencies]
10 bytes = "1.5.0"
11 futures = "0.3.29"
12 tokio = { version = "1.33.0", features = ["full"] }
13 tokio-util = { version = "0.7.10", features = ["full"] }
```

`cargo add` 工具为我们准确配置了具体依赖库的版本号和特性。

基于 tokio 实现 tcp server

我们的 tcp server 实际要干下面几件事儿。

1. 接收 tcp client 的连接，每一个新连接创建一个新的 task。
2. 读取 tcp client 发过来的指令数据。
3. 根据指令，获取服务器本地的时间信息。
4. 将得到的信息字符串写入 socket，返回给客户端。

[复制代码](#)

```
1 use std::env;
2 use tokio::io::{AsyncReadExt, AsyncWriteExt};
3 use tokio::net::TcpListener;
4 use tokio::process::Command;
5
6 #[tokio::main]
7 async fn main() -> Result<(), Box<dyn std::error::Error>> {
8     let addr = env::args()
9         .nth(1)
10        .unwrap_or_else(|| "127.0.0.1:8888".to_string());
11    println!("Listening on: {}", addr);
12    let listener = TcpListener::bind(&addr).await?;
13
14    // 注意这里是一个无条件循环，表明始终处于服务状态
15    loop {
16        // 等待客户端请求连上来
17        let (mut socket, _) = listener.accept().await?;
18
19        // 来一个客户端连接，创建一个对应的新任务
20        tokio::spawn(async move {
21            // 分配一个缓冲存
22            let mut buf = [0; 1024];
23            let mut offset = 0;
24            // 循环读，因为不能确保一次能从网络线路上读完数据
25            loop {
26                // 读操作，返回的n表示读了多少个字节
27                // 正常情况下，读到数据才会返回，如果没有读到，就会等待
28                let n = socket
29                    .read(&mut buf[offset..])
30                    .await
31                    .expect("failed to read data from socket");
32
33                // n返回0的情况，是碰到了EOF，表明远端的写操作已断开，这个一定要判断
34                if n == 0 {
35                    // 碰到了EOF就直接返回结束此任务，因为后面的操作没了意义
```

```

36         return;
37     }
38
39     println!("offset: {offset}, n: {n}");
40     let end = offset + n;
41     // 转换指令为字符串
42     if let Ok(directive) = std::str::from_utf8(&buf[..end]) {
43         println!("{directive}");
44         // 执行指令对应的工作
45         let output = process(directive).await;
46         println!("{output}");
47         // 向客户端返回处理结果
48         socket
49             .write_all(&output.as_bytes())
50             .await
51             .expect("failed to write data to socket");
52     } else {
53         // 判断是否转换失败，如果失败，就有可能是网络上的数据还没读完
54         // 要继续loop读下一波数据
55         offset = end;
56     }
57 }
58 });
59 }
60 }
61
62 async fn process(directive: &str) -> String {
63     if directive == "gettime" {
64         // 这里我们用了unwrap()是因为我们一般确信执行date命令不会失败
65         // 更可靠的做法是对返回的Result作处理
66         let output = Command::new("date").output().await.unwrap();
67         String::from_utf8(output.stdout).unwrap()
68     } else {
69         // 如果是其他指令，我们目前返回 无效指令
70         "invalid command".to_owned()
71     }
72 }

```

代码中有详细解释，这里我也补充说明一下。

首先，我们给 main 函数指定了返回类型：`Result<(), Box<dyn std::error::Error>>`。错误类型部分是一个用 `Box<T>` 装盒的 trait object，这个 trait object 是针对标准库中的 Error trait 的。在这里的意思是，凡是实现了 Error trait 的类型都可以作为错误类型从 main 函数中返回。

第 12 行 `let listener = TcpListener::bind(&addr).await?`；行末有一个问号，这是 Rust 里的问号操作符，意思是 `await` 得到数据后是一个 `Result<>`，如果这个 `Result<>` 是 `Ok` 值，那就解开它，返回里面的内容，这个例子里是一个 `TcpListener` 实例；而如果这个 `Result<>` 是 `Err` 值，那就直接从函数中返回，不再往下执行。问号操作符在这里起一个防御式编程的作用，能够让流程代码显得更简洁。

注意第 15 行是一个 `loop` 无条件循环，也就是死循环。为什么呢？因为这是个服务端程序，是需要一直跑着的，退出就意味着出问题了。

第 17 行，监听客户端来的连接，来一个就产生一个 `socket` 实例。我们看到，在 `let` 后面用了模式匹配写法，直接把元组析构了。如果来了多个连接，就会产生多个 `task`，它们之间互不干扰，是并发处理的。

第 20 行，针对每一个连上的客户端连接，创建一个新的 `tokio` 轻量级线程 `task`，来处理对应的任务。继续往下，第 22 行，可以看到，这个服务端程序为每个连接创建了一个缓冲区，大小是 1024 字节。从网络上读到的数据会放在这个缓冲区里面。

第 25 行，再次用了一个循环。因为网络上的数据是呈流的形式过来的，在一次 CPU 读取它之前，这些数据有可能还没完全到达服务器上面。因此可能需要多次读。读没读够，可以尝试把已经读到数据转换成字符串，看是否能成功来判断（这个判断方式并不严谨，这里主要用于说明流程）。如果成功了，就调用 `process()` 业务函数来计算。

`process()` 异步函数中使用了 `tokio::process::Command` 类型来调用系统中的 `date` 命令，这是一个 Linux 下的查看系统日期时间的命令，会输出下面这种格式：

 复制代码

```
1 Tue Oct 31 14:56:27 CST 2023
```

注：如果你使用 Windows 的话，可以找找 Windows 里的替代命令。

`process()` 函数会返回这个字符串。

然后，通过同一个 socket，将数据返回给客户端连接：`socket.write_all`。

在多次读的过程中，要注意偏移量 offset 的处理。可以看到，代码量虽然不多，但是充满了细节，请你仔细品味一下。

基于 tokio 实现 tcp client

下面我们来看对应的 tcp 客户端应该怎么实现。

因为我们要马上再创建一个可执行程序，所以默认的 `cargo run` 命令就不能满足这个需求，它默认只能启动一个二进制文件。我们需要改一下 Cargo.toml 的配置，在文件中加入一些内容。

[复制代码](#)

```
1  [[bin]]
2  name = "server"
3  path = "src/server.rs"
4
5  [[bin]]
6  name = "client"
7  path = "src/client.rs"
```

然后把 src 目录下的 main.rs 改成 server.rs，并创建一个新文件 client.rs，代码如下：

[复制代码](#)

```
1  use std::env;
2  use tokio::io::{AsyncReadExt, AsyncWriteExt};
3  use tokio::net::TcpStream;
4
5  #[tokio::main]
6  async fn main() -> Result<(), Box<dyn std::error::Error>> {
7      let addr = env::args()
8          .nth(1)
9          .unwrap_or_else(|| "127.0.0.1:8888".to_string());
10     // 连接到服务端
11     let mut stream = TcpStream::connect(&addr).await?;
12
13     // 写入指令数据，这是一种最简单的协议。
14     stream.write_all(b"gettime").await?;
```

```

15
16 // 等待tcp server的回复, 读取内容
17 // 这里用动态数组来存储从服务端返回的内容
18 let mut buf: Vec<u8> = Vec::with_capacity(8128);
19 // 读取的缓冲区
20 let mut resp = [0u8; 2048];
21 loop {
22     // 尝试一次读, 返回读到的字节数
23     let n = stream.read(&mut resp).await?;
24     // 将读到的字节合并到buf中去
25     buf.extend_from_slice(&resp[0..n]);
26     if n == 0 {
27         // 流断掉了
28         panic!("Unexpected EOF");
29     } else if buf.len() >= 28 {
30         // like: "Tue Oct 31 14:56:27 CST 2023"
31         // buf 已经填充了足够的内容
32         break;
33     } else {
34         // buf 中还没有足够多的内容, 继续填充...
35         continue;
36     }
37 }
38
39 // 转换并打印返回的信息
40 let timeinfo = String::from_utf8(buf)?;
41 println!("{}", timeinfo);
42
43 Ok(())
44 }

```

代码中有详细解释, 这里我也做一下补充说明。

第 11 行, 我们使用 `TcpStream::connect()` 连到服务端上去, 注意这点和服务端的监听是不同的。然后第 14 行, 向 tcp 连接中写入协议指令, 这里就是简单的字节串:

`b"gettime"`。

然后第 17 行到 25 行, 我们采用了另外一种方式来存储读回来的数据, 这里用到了动态数组 `Vec` 的 API `extend_from_slice()`, 这个的好处是不需要由自己来维护每次读到的偏移量。第 21 行, 我们再次使用了 `loop`, 还是同样的原因, 网络上的数据流, 我们有可能一次读取不完, 需要多次读才行。

第 29 行是跳出此循环的判断条件，这里是因为我们知道会返回 date 命令的输出结果，它是固定的 28 个字节的长度。这个条件相当死板，这里也只是起演示作用。

最后就把这个从服务端程序得到的内容打印出来。下面我们开始测试。

测试

编译运行服务端：

```
1 cargo run --bin server 或 cargo run --bin server -- 127.0.0.1:8888
```

 复制代码

编译执行客户端：

```
1 cargo run --bin client 或 cargo run --bin client -- 127.0.0.1:8888
```

 复制代码

查看执行效果。

服务端打印：

```
1 Listening on: 127.0.0.1:8888
2 offset: 0, n: 7
3 gettime
4 Tue Oct 31 15:04:08 CST 2023
5
```

 复制代码

客户端打印：


```
1 Tue Oct 31 15:07:48 CST 2023
2
```

这样，我们就成功实现了我们的第一个命令行应用，并完整体验了 tokio 网络编程。

Frame 层

前面我们的实现，完成了这个应用初步的功能。但是实际上对于我们初学者来说，难度还是比较大，主要体现在 4 个方面。

1. 什么时候要加 loop，什么时候不加？
2. 两个端的合法性判定条件是什么？能否推广到适用面更广的方式？
3. 多次读取的偏移量或缓冲区如何准确维护？
4. tcp 网络协议的背景知识，服务端与客户端的基本概念，EOF 在什么条件下触发等等。

其中前面三条都属于实现层面的细节，我们可以来了解一下产生这些复杂性的原因。

复杂性的来源

我们知道，一个 tcp 连接就像一个管道一样，里面流过的是一个个的字节，也就是说，它传输的是一个字节流。然而，由于网络本身的复杂性，比如如果服务器在很远的地方，数据有可能要经历过很多次路由器 / 交换机才能到达，有可能一个字节和下一个字节之间会间隔一段时间才能传过来。这时候，直接在 socket 上做的一次 read，并不能保证就一定读完整了我们想要的内容，有可能只读到了一个片段而已。

这个问题其实属于传输层的问题，不应该让上层业务开发人员来担忧。如果在考虑业务的同时，还要考虑这种底层问题的话，总会感觉施展不开，各种细节会缠得你寸步难行。

比如有两个人 A 和 B，A 给 B 发送两条消息 msg_a 和 msg_b，B 接收消息的时候，一般会希望编程接口每次拿到一个完整的消息，第一次取是 msg_a，不多也不少；第二次取是

msg_b，不多也不少。不会担心每次取的时候，消息可能取不完整，需要用一个循环重复取，并且还得把每次取到的片段拼接成一个大的完整的消息内容。

当消息体小的时候，可能问题不明显，当消息体大的时候，比如一次有几兆内容的时候，这个问题其实就无法忽略了。

好在 tokio 框架已经为我们考虑了这个问题。tokio 生态引入了 Frame 的概念，Frame 就是一个帧 / 框，一个 Frame 里可以包含一段完整的可预期的信息。相当于它在 tcp 这一层之上又抽象了一层，封装了具体怎么读取和切分原始的字节序列这个问题。Frame 让我们读到的总是业务关心的一批批数据：msg。Frame 机制将网络流的原始字节序列转换成 Frame 序列，并且会自动帮我们确定 Frame 的边界在哪里。

Frame 编解码

既然是 Frame，就涉及到采用何种编码的问题。Frame 本身只是一个框的概念，这个框里面具体填什么格式的内容，是由编码决定的，写入的时候编码，取出的时候需要解码。

与 tokio 配套的 tokio_util crate 里，提供了四种最简单的编解码类型：BytesCodec、LinesCodec、AnyDelimiterCodec、LengthDelimitedCodec。我们分别介绍一下。

BytesCodec

Frame 长度为 1 的编解码。一个字节一个 Frame。适用于文本和二进制的任何网络协议。如果你的应用就想一个字节一个字节地处理数据流，这个就是合适的选择。

LinesCodec

行编解码协议，它使用 `\n` 作为 Frame 之间的分隔。这个协议用得很多，特别适合文本网络协议。

AnyDelimiterCodec

指定间隔符的编解码协议，这个相当于 LinesCodec 的扩展版本。用这个协议，你可以自定义任何 Frame 分隔符，比如 ; , # 等等。它也比较适用于文本网络协议。

LengthDelimitedCodec

长度分隔编解码协议。这个协议的思路是，每一个 msg 发送的时候，在它前面，都加上一个固定位数的长度表示前缀。这样，每次读到这个前缀数字的时候，就知道接下来要读多少个字节，才会形成一个完整的 Frame。比如编码后的一个 Frame 类似下面这个样子：

[复制代码](#)

```
1 +-----+-----+
2 | len: u32 |           frame payload           |
3 +-----+-----+
```

这个方法是协议设计的典型方法，具有非常强的通用性，适用于文本或二进制的网络协议。

tokio 不仅提供了这些默认的简单的编解码方案，它还允许你自定义 Codec。如果上面 4 种都不能满足你的需求，你完全可以按它的规范自定义一个。

此处，我们可以用 LengthDelimitedCodec，只需要在基本的 TcpStream 上套一下就可以了。接着往下看。

使用 Frame 改造代码

使用 Framed + LengthDelimitedCodec 类型改造后的服务端和客户端代码如下：

服务端代码：

[复制代码](#)

```
1 use bytes::Bytes;
2 use futures::{SinkExt, StreamExt};
3 use std::env;
4 use tokio::net::TcpListener;
5 use tokio::process::Command;
6 use tokio_util::codec::{Framed, LengthDelimitedCodec};
```

```

7
8 #[tokio::main]
9 async fn main() -> Result<(), Box<dyn std::error::Error>> {
10     let addr = env::args()
11         .nth(1)
12         .unwrap_or_else(|| "127.0.0.1:8888".to_string());
13     println!("Listening on: {}", addr);
14     let listener = TcpListener::bind(&addr).await?;
15
16     // 注意这里是一个无条件循环，表明始终处于服务状态
17     loop {
18         // 等待客户端请求连上来
19         let (stream, _) = listener.accept().await?;
20         // 包裹成一个Frame stream
21         let mut framed_stream = Framed::new(stream, LengthDelimitedCodec::new());
22
23         // 创建子task执行任务
24         tokio::spawn(async move {
25             // 等待读取一个一个msg，如果返回None，会退出这个循环
26             while let Some(msg) = framed_stream.next().await {
27                 match msg {
28                     Ok(msg) => {
29                         // 解析指令，执行任务
30                         let directive = String::from_utf8(msg.to_vec())
31                             .expect("error when converting to string directive.");
32                         println!("{directive}");
33                         let output = process(&directive).await;
34                         println!("{output}");
35
36                         // 返回执行结果
37                         _ = framed_stream.send(Bytes::from(output)).await;
38                     }
39                     Err(e) => {
40                         println!("{e:?}");
41                     }
42                 }
43             }
44         });
45     }
46 }
47
48 async fn process(directive: &str) -> String {
49     if directive == "gettime" {
50         // 这里我们用了unwrap()是因为我们一般确信执行date命令不会失败
51         // 更可靠的做法是对返回的Result作处理
52         let output = Command::new("date").output().await.unwrap();
53         String::from_utf8(output.stdout).unwrap()
54     } else {
55         // 如果是其他指令，我们目前返回 无效指令

```

```

56         "invalid command".to_owned()
57     }
58 }

```

这里我简单解释一下上面的示例。

在监听到连接 stream（第 19 行）后，把它包裹成 Frame stream（第 21 行），然后使用 `while let` 配合 `framed_stream.next()` 对这个流进行迭代，就读出了里面一帧一帧的数据 msg。需要返回结果的时候，使用 `framed_stream.send()` 就可以了。

客户端代码：

 复制代码

```

1  use bytes::Bytes;
2  use futures::{SinkExt, StreamExt};
3  use std::env;
4  use tokio::net::TcpStream;
5  use tokio_util::codec::{Framed, LengthDelimitedCodec};
6
7  #[tokio::main]
8  async fn main() -> Result<(), Box<dyn std::error::Error>> {
9      let addr = env::args()
10         .nth(1)
11         .unwrap_or_else(|| "127.0.0.1:8888".to_string());
12     // 连接到服务端
13     let stream = TcpStream::connect(&addr).await?;
14     // 包裹成 Frame stream
15     let mut framed_stream = Framed::new(stream, LengthDelimitedCodec::new());
16
17     // 发送指令
18     framed_stream.send(Bytes::from("gettime")).await?;
19
20     // 读取返回数据，这里只读一次
21     if let Some(msg) = framed_stream.next().await {
22         match msg {
23             Ok(msg) => {
24                 let timeinfo = String::from_utf8(msg.to_vec())?;
25                 println!("{}", timeinfo);
26             }
27             Err(e) => return Err(e.into()),
28         }
29     }

```

```
30     Ok(()))
31 }
32
```

在连接到服务器，得到连接 stream（第 13 行）后，把它包裹成 Frame stream（第 15 行），然后使用 `framed_stream.send()` 发送一条指令，在后面用迭代器方法等待指令执行后返回的内容 msg。对 msg 的处理方式和服务端代码一致。

可以看到，经过 Frame 层抽象后，大大精简了代码逻辑，整体变得非常清爽。最关键的是，改造后的代码实际**完全重塑了程序员的心智模型**：我们不再需要关注底层传输的大量细节了，真正实现了面向业务编码，可以按时下班了，非常开心。

注：这节课的完整代码点击 [🔗 这里](#) 可以获取。

小结

这节课我们一起学习了如何循序渐进地基于 tokio 开发一个服务端和客户端 tcp 网络应用程序。知识点很多，我们一起来回顾一下。

Rust 中命令行参数的获取方式；

在 Cargo.toml 配置文件中添加依赖；

在服务端建立 tcp lisener；

根据新的连接创建新的 task；

读取 tcp 数据输入，以及写入返回内容；

建立 tcp client 连接；

编译测试 Rust 命令程序；

tokio 的 Frame 概念和编解码；

使用 Frame 简化网络编程。

网络编程有其复杂性在里面，所以这节课我们应该重点掌握使用 Frame 的编程模型，同时了解原始字节流的处理原理。

思考题

最后请你来思考 2 个问题。

1. EOF 是什么，什么时候会碰到 EOF?
2. 请问 `stream.read_to_end()` 接口能读完网络连接中的数据吗?

欢迎你把思考后的结果分享到评论区和我们一起讨论，也欢迎你把这节课的内容分享给其他朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (12)

最新 精选



PETFiSh

2023-11-20 来自四川

EOF: End of file. 在linux万物皆file的情况下connection也可以是一个file。所以，当Connection关闭的时候，就会产生EOF。

`stream.read_to_end()`是持续`read()`直到EOF，因此能够读完网络里的数据，如果使用`stream.read_to_end(&mut buf).await`；读取的话，会持续wait，直到连接关闭才能进行后续的操作。

作者回复: 👍，真棒！



👍 2



学水

2023-11-20 来自加拿大

感觉这部分内容和java的netty包有点像

作者回复: 对的，其实网络编程到了底层基本都是这个样子了。

共 2 条评论 >

👍 2



Citroen

2023-11-24 来自辽宁

这段代码

```
let mut framed_stream = Framed::new(stream, LengthDelimitedCodec::new())
```

请问一下老师，我现在需要接收client端的数据（数据是打包好的protobuf序列化后的二进制数据，

前面有四位长度表示这个序列化后的二进制数据的长度，也就是说数据格式是，头（四位 用于说明后面完整protobuf序列化后的数据长度）加内容（protobuf序列化后的二进制数据），len(四位长度) 加main(protobuf序列化后的二进制数据 每次长度不固定，但可通过前面的len可以知道)，每段数据都是这样 这样的数据怎么取？

作者回复: 很简单，发送端用 LengthDelimitedCodec 编码的，接收端也同样用 LengthDelimitedCodec 解码就行了。tokio的这层Framed包装自动完成了这项工作了，具体的编解码是在下层layer做的，上层你就关心业务就好啦，已经给你取到了。

共 3 条评论 >

👍 1



哄哄

2023-11-20 来自北京

应该用什么已有的crate包，感觉也是新人的一个问题。请问上哪能快速认识常用第三方包呢？

作者回复: crates.io, lib.rs, <https://github.com/rust-unofficial/awesome-rust>



👍 1



沉默的话唠

2023-12-28 来自广西

跟着一路下来，感觉讲师讲的真的不错。不知道后面考不考虑有训练营之类的，更加全面体系的课程。

作者回复: 感谢肯定 🙏，回头和小编商量商量。



superggn

2023-12-21 来自北京

EOF => End Of File, 在数据流断开的时候会碰到

`stream.read_to_end(buf)` => 把这个 stream 里接收到的所有数据都丢到一个 `buf_vec` 里, 应该是一个阻塞函数吧, 只要链接不断开, 就一直等待, 单开一个进程 / 线程来跑 `read_to_end` 然后在别的进程 / 线程从 `buf` 里往外面读数据

不过感觉如果用 `read_to_end` 的话当前 stream 就没办法往里写数据了, 这咋整啊, 难道是 clone 一份 stream 然后在其他进程 / 线程来写吗, 还是设计的时候就不考虑往 `用了 `read_to_end` 的 stream` 里继续写数据了?

作者回复: tokio stream是全双工的, 可以写。但是你如果程序被卡在 `read_to_end` 那里, 那写逻辑也执行不了。



superggn

2023-12-21 来自北京

啊? 有源码的啊, star 一下先

作者回复: <https://github.com/miketang84/jikeshijian/>



-Hedon

2023-12-20 来自湖北

思考题1: EOF

EOF 是 "End of File" 的缩写, 意为“文件结束”。它是一个用于表示文件或数据流末尾的控制信号或字符。当读取文件或数据流时, EOF用于指示没有更多的数据可读取。

EOF 通常出现在以下几种情况:

1. 文件读取: 在读取文件内容时, 到达文件末尾后, 会遇到 EOF。这告诉程序没有更多的数据可读取。

2. 标准输入: 在某些编程语言或环境中, 如 Unix/Linux 的命令行, 可以使用特定的键盘组合 (如 `Ctrl+D`) 来生成 EOF 信号, 表示标准输入结束。

3. 数据流结束：在处理数据流（如网络传输中的数据）时，EOF 可以用来指示数据传输已经完成。

思考题2：stream.read_to_end()

首先我觉得核心点在“读完”这个概念上，即意味着连接中的数据是有限的，才有读完，如果是无限的流数据，那么 stream.read_to_end() 会一直阻塞等待数据的到来。如果明确数据的有限的，那么在数据源发出 EOF 后，stream.read_to_end() 就会返回，那这个时候是可以“读完”的。当前，前提是你的内存得够。

(连续2周搬砖赶需求，终于能抽点时间续上 rust 的学习了 T_T)

作者回复：👍，不错。



A0.何文祥

2023-11-20 来自广东

cargo new --bin getinf应为cargo new --bin getinfo

编辑回复：再次感谢👉



Andylinge

2023-11-20 来自江苏

Windows 11 报如下错误，大家可以把process里面的获取时间命令自己用Rust重写一下。

```
```Listening on: 127.0.0.1:8880
```

```
Accepted a connection from: 127.0.0.1:9354
```

```
gettime
```

```
thread 'tokio-runtime-worker' panicked at src/server.rs:80:58:
```

```
called `Result::unwrap()` on an `Err` value: Error { kind: NotFound, message: "program not found" }
```

```
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

```
error: process didn't exit successfully: `D:\00-Program\05-Rust\14-Rust_MikeTang\r14_tokio
```

```
\getinfo\target\debug\server.exe` (exit code: 0xc000013a, STATUS_CONTROL_C_EXIT)
` ``
```

作者回复: 对的，date命令是linux下的，抱歉忘了这个细节，后面想办法补充说明一下。

共 2 条评论>

